ROY CONSULTANCY

Software Requirements Specification

A developer-ready functional blueprint for a Legal & Compliance Practice Management System

Purpose

This document defines the required modules, functional behaviour, core data entities, workflows, access controls, technical expectations, and acceptance criteria for building Roy Consultancy's internal backend application. It is intended to be supplied to an AI coding agent or software development team as the primary implementation brief.

Field	Details
Document type	Software Requirements Specification (SRS)
Product	Roy Consultancy Operations Platform
Business domain	Business registrations, tax registrations, corporate compliance, intellectual property and related services
Suggested architecture	Modular monolith
Suggested stack	Next.js, NestJS, TypeScript, PostgreSQL, Prisma, Redis/BullMQ, private object storage
Version	1.0
Prepared	10 October 2026

Important: This is a product and engineering specification, not legal advice. Statutory requirements, service eligibility, professional authorisations, tax treatment and deadlines must be verified against current official sources by an appropriately qualified person before production use.

1. Document purpose and implementation instructions

The application should centralise Roy Consultancy's client lifecycle and service delivery. It must manage enquiries, client/business records, service orders, cases, documents, compliance obligations, communications, billing, staff workload and management reporting.

Instructions for the implementation agent

- Read the full specification before creating the project structure. Produce a brief implementation plan and identify any conflicts or missing decisions before making irreversible architectural choices.
- Implement a secure, working application—not only static screens, mock data, or a UI prototype. Every visible action must call a real backend operation and persist valid data.
- Use migrations and seed data. Include sample service templates and clearly marked demonstration records, but do not mix demo data with production data.
- Use reusable, configurable workflows and service templates instead of hardcoding separate case-management logic for every service.
- Implement modules incrementally in the priority order specified in this document. Maintain automated tests and update API documentation as endpoints change.
- Never invent government API access, automated filing capability, legal rules, statutory deadlines or professional approvals. Where no authorised integration exists, support manual tracking and evidence uploads.
- Before declaring a module complete, demonstrate its end-to-end workflow, validation, authorization, audit history and error handling.

Product goals

- Create one reliable record for every client, business entity, service order, case, obligation, document and financial transaction.
- Make deadlines, pending client inputs, rejected submissions, outstanding invoices and unassigned tasks visible and actionable.
- Reduce manual follow-up and spreadsheet dependency through configurable reminders, task queues and dashboards.
- Preserve a traceable history of operational decisions, approvals, filing evidence and financial changes.
- Support growth in the number of clients, entities, services and staff without redesigning the core data model.

Out of scope for the first release

- Direct government portal submission unless an official, authorised and technically supported integration is separately specified.
- Replacing a full general ledger/accounting package or providing legal/tax advice automatically.
- Autonomous AI decisions on eligibility, legal interpretation, statutory deadlines or filing approval.
- A public SaaS multi-tenant product for unrelated consultancies. The initial product is for Roy Consultancy; tenant isolation is only needed if multi-firm SaaS is later approved.

2. Contents

3. Product scope and user journeys

4. Roles and permission model

5. Functional modules (Modules 1–16)

6. Service catalogue and initial service templates

7. End-to-end workflows and state transitions

8. Data model and database requirements

9. API and integration standards

10. Security, privacy and audit requirements

11. Reporting and dashboards

12. Non-functional requirements

13. Testing and acceptance criteria

14. Implementation phases and priority

15. Definition of done

Appendix A. Suggested repository structure

Appendix B. Agent handoff checklist

Priority definitions

Priority	Meaning
P0 — MVP	Required for the first production release; build and test before launch.
P1 — Next	Important follow-up capability; design the core interfaces now but it may ship after MVP.
P2 — Later	Expansion capability that must not delay a stable first release.

3. Product scope and user journeys

Primary user journeys

- **Lead to client:** create lead → contact/follow-up → qualify → prepare quotation → record acceptance → create or link client and business entity.
- **Service delivery:** choose service → create service order/case → assign owner/reviewer → generate checklist/tasks → collect and review documents → prepare and submit → track external response → deliver outcome → close case.
- **Recurring compliance:** identify an applicable obligation → create obligation for the relevant entity and period → assign owner → remind client/team → record filing details and evidence → close obligation → generate the next period when applicable.
- **Billing:** create quotation → invoice according to configured policy → record payment → allocate payment → calculate balance → follow up on overdue amount → issue approved credit note/refund if needed.
- **Management:** view dashboard → identify overdue/high-risk work → reassign/escalate → inspect history → measure completion, collection and workload.

Core business rules

- A client may represent or own multiple business entities. A business entity may have multiple directors, partners, proprietors, authorised representatives or other members.
- A client may request multiple services; each service order can create one or more cases if the service definition requires separate workstreams.
- A case must reference a client, service version and, where applicable, a business entity. The case must preserve the service/template version used at creation.
- A compliance obligation is not the same as a case. An obligation represents a requirement for a business entity and period; a case represents work being performed to fulfil a service request.
- A statutory due date, internal target date, expected completion date, submission date and actual completion date must be stored separately.
- Financial records must not be silently overwritten. Corrections should use appropriate adjustment, credit-note, refund or reversal records with permissions and audit history.
- Use explicit timezone configuration for displayed dates. Store timestamps consistently in UTC; date-only statutory deadlines should be represented as date values where appropriate.
- Every critical workflow action must have clear validation, authorization and an audit event.

4. Roles and permission model

Implement role-based access control (RBAC) with granular permissions. Roles are starting templates, not a substitute for checking permission on each endpoint and record.

Role	Default responsibilities	Restrictions / notes
Super Admin	System configuration, user/role administration, integrations, audit and operational settings.	Highly privileged; sensitive actions audited.
Business Owner	Business-wide operations, finance overview, approvals, service catalogue and reports.	May be separate from technical Super Admin.
Operations Manager	Case allocation, task queues, deadlines, escalations and quality review.	Finance and identity-document access only when granted.
Sales Executive	Leads, follow-ups, quotations and assigned client details.	No unrestricted access to sensitive documents or finance administration.
Compliance Executive	Assigned cases, checklists, documents, filing tracking and client updates.	Cannot approve own high-risk work if maker-checker is required.
Reviewer / Approver	Review documents and cases, request corrections, approve defined transitions.	Approval scope limited by service/permission.

Role	Default responsibilities	Restrictions / notes
Finance Executive	Invoices, receipts, payments, allocations, approved refunds and receivables.	No unnecessary access to full identity documents.
Client / Client Representative	Own/authorised profile, business entities, cases, requested uploads and invoices.	Must not access another client's records; authority may be entity-specific.
Auditor / Read-only	Explicitly scoped read-only records and reports.	No create/update/delete or financial execution permissions.

Permission design requirements

- Permissions should be action-based, e.g. clients.read, clients.update, documents.download, cases.assign, cases.transition, invoices.issue, payments.record, refunds.approve, reports.finance.read, users.manage.
- Apply object-level authorization after loading each record. Do not rely on hidden UI controls.
- Scope access by assignment, department, client, entity, case or business-wide privilege where needed.
- Support separation of duties for configured high-risk actions such as refunds, final filing approval and permission changes.
- A client representative's access must be tied to explicit client/entity authorisation and be revocable.
- Audit role changes, permission changes, user disablement, access to sensitive documents and privileged actions.

Module 1. Authentication, users and access control

Purpose: Securely identify users and enforce access across all modules.

Functional requirements

- Login, logout, session renewal and session revocation; secure password reset and account invitation flows.
- Support secure password hashing using a modern algorithm; never store plaintext passwords.
- Optional or required MFA for privileged users, configurable by policy.
- User profile, status (invited/active/suspended/disabled), department, manager and last-login information.
- Role and permission management with least-privilege defaults.
- Rate-limit authentication endpoints; monitor repeated failures and suspicious activity.
- Validate authentication and authorization on every API request; use secure, HttpOnly, SameSite cookies if cookie-based sessions are selected.
- Provide administrative ability to revoke all sessions after role change, account compromise or termination.

Acceptance criteria

- Disabled users cannot authenticate or continue using revoked sessions.
- Every protected endpoint rejects unauthenticated or unauthorized requests.
- Role/permission changes take effect within a documented, short period and are audited.

Module 2. CRM, leads and client management

Purpose: Manage prospective clients, contact history, conversion and ongoing relationships.

Functional requirements

- Create, update, search, filter and archive leads; capture source, service interest, owner, priority, stage and next follow-up.
- Record calls, meetings, emails, notes, tasks, reminders and outcomes in a chronological activity timeline.
- Support lead stages such as New, Contacted, Qualified, Quotation Sent, Won, Lost and Nurture; make stages configurable.
- Convert a lead to a client without losing history; link an existing client rather than creating duplicates.
- Store client type, legal name, preferred name, contact channels, billing details, communication preferences and status.

- Support individual clients and organisation clients. Separate personal contact records from business-entity records.
- Provide duplicate warnings based on normalized email/phone and other suitable identifiers; allow authorised merge with audit history.
- Capture referral source, campaign, assigned sales executive, expected value, loss reason and follow-up date.
- Display all associated business entities, cases, quotations, invoices, documents (subject to permission) and communications.
- Support client tags, internal notes and configurable custom fields without using uncontrolled JSON for core business fields.

Acceptance criteria

- A lead can be converted or linked to an existing client while preserving activity history.
- Search and filters work across permitted records and do not leak restricted data.
- Duplicate merge preserves references and writes an audit event.

Module 3. Business entities and relationship management

Purpose: Maintain structured profiles for the companies, LLPs, proprietorships, partnerships, trusts and other entities served.

Functional requirements

- Store entity type, legal name, trade name, incorporation/formation date, jurisdiction, status, addresses and relevant identifiers.
- Support entity-specific contacts, directors, designated partners, partners, proprietors, trustees, authorised signatories and other configurable roles.
- Record relationship start/end dates and authority to act for the entity where applicable.
- Store registered office, principal place of business, mailing address and historical addresses with effective dates.
- Record identifiers such as CIN/LLPIN, GSTIN, PAN or other relevant identifiers only where needed for the service and with restricted access.
- Allow one client to manage multiple entities and one entity to have multiple authorised contacts.
- Attach entity documents and link them to cases/obligations without unnecessary duplication.
- Track relevant entity dates and status changes; keep history rather than overwriting material historical facts.

Acceptance criteria

- Cases and compliance obligations can be associated with the correct entity.
- An entity’s authorized client representatives can be managed and revoked independently.
- Entity identifiers are masked in ordinary list views where appropriate.

Module 4. Service catalogue, pricing and service templates

Purpose: Define what Roy Consultancy sells and how each service is delivered.

Functional requirements

- Create categories, subcategories, services, variants, packages, add-ons and service descriptions.
- Define service eligibility notes, exclusions, expected turnaround, dependencies, required client inputs and applicable jurisdiction.
- Version service definitions and pricing with effective dates; existing orders retain the version and price used at sale.
- Separate professional fees, government/statutory fees, third-party charges, discounts, taxes and reimbursable expenses.
- Configure one-time services, recurring retainers, annual packages and optional instalment schedules.
- Set default document requirements, task templates, review gates, status transitions, deliverables and closure criteria.

- Allow authorised users to create client-specific quotations while preserving standard price references and discount approvals.
- Mark services as draft, active, paused or retired; retired services remain visible on historical cases.
- Include service owner, review cadence and last-verified date for operational/compliance guidance.

Acceptance criteria

- A service can be configured without code changes for ordinary workflow variations.
- Historical cases are not altered when service prices/templates are updated.
- Pricing calculations are deterministic and covered by tests.

Module 5. Quotations, orders and service engagement

Purpose: Convert accepted commercial proposals into trackable work.

Functional requirements

- Create quotations with unique number, client, entity, line items, scope, exclusions, validity, taxes, discount and payment terms.
- Support quotation statuses: Draft, Sent, Viewed (if supported), Accepted, Rejected, Expired and Cancelled.
- Record acceptance date, acceptance method and evidence/consent where appropriate.
- Support revisions while preserving prior versions and change history.
- Convert an accepted quotation into a service order and one or more cases without duplicate creation.
- Allow manual case creation for authorised exceptions, requiring a reason and audit entry.
- Record advance/deposit requirements and link to invoices/payments without treating a quotation as a paid invoice.
- Define cancellation, refund review and scope-change workflows; changes to scope should be represented explicitly.

Acceptance criteria

- Repeated conversion requests do not create duplicate orders/cases.
- The accepted quotation version, pricing and scope are retained against the order.
- Case creation is blocked when required commercial approvals are missing, according to configuration.

Module 6. Case, task and workflow management

Purpose: Coordinate the full delivery lifecycle for every service request.

Functional requirements

- Generate unique case reference, service/client/entity links, owner, reviewer, priority, opened date, target date and status.
- Support configurable service-specific state machines, transition guards and required fields.
- Default statuses may include New, Awaiting Payment, Awaiting Client, Documents Under Review, In Preparation, Internal Review, Ready to Submit, Submitted, Awaiting Authority, Correction Required, Approved, Delivered, Completed, On Hold and Cancelled.
- Create tasks from templates with assignee/role, due date, priority, dependencies, estimated effort, checklist and completion evidence.
- Support reassignment, escalation, task comments, internal notes and client-visible updates as distinct content types.
- Record external application/acknowledgement reference, portal name, submission timestamp, response date and outcome.
- Support rejection/resubmission loops without deleting previous submission attempts or evidence.
- Provide case timeline, status history, task board/list, search, filters, bulk actions (permission-controlled) and saved views.
- Track pause reason and pause/resume timestamps. Do not silently erase elapsed time or legal due dates.
- Require closure checklist and final deliverable(s) before completion; allow authorised reopen with reason.

- Support SLA rules for internal targets and escalation; label these separately from statutory deadlines.

Acceptance criteria

- Invalid transitions and missing prerequisites are rejected by the backend.
- Every status transition captures actor, timestamp, prior/new status and reason where required.
- Case closure requires configured completion evidence and checklist items.
- Reopened cases retain the original history.

Module 7. Document vault, checklists and review

Purpose: Securely collect, validate, version and deliver client documents.

Functional requirements

- Store document metadata in PostgreSQL and binary files in private object storage.
- Support document type, owner/client/entity/case links, source, received date, issue/expiry date, version, verification status and sensitivity classification.
- Generate required-document checklists from service templates and allow permitted case-specific additions.
- Support statuses such as Requested, Uploaded, Under Review, Accepted, Rejected, Expired and Superseded.
- Record reviewer, review timestamp, rejection reason and client re-upload request.
- Preserve document versions and prevent silent replacement of a reviewed file.
- Use short-lived signed URLs, authorization checks, file type/size limits, malware scanning and safe preview/download behaviour.
- Restrict access to highly sensitive identity documents separately from general case attachments.
- Record sensitive document views/downloads and sharing events; never include file contents or secret links in logs.
- Support secure deliverable packages and expiring client download links where appropriate.
- Provide retention labels, legal hold where needed, and controlled archival/deletion workflows.

Acceptance criteria

- Files cannot be retrieved by guessing a file ID or using an expired/revoked URL.
- A rejected document has a recorded reason and previous versions remain traceable.
- The application blocks prohibited file types and handles scanning failures safely.

Module 8. Compliance obligations, calendar and renewals

Purpose: Track recurring and date-sensitive duties separately from individual service cases.

Functional requirements

- Create obligations linked to an entity, obligation type, period/financial year, applicability rationale and governing rule version.
- Store statutory due date, internal target, expected completion, submission date, completion date, status and assigned owner/reviewer separately.
- Support statuses such as Not Assessed, Not Applicable (with reason), Upcoming, In Progress, Awaiting Client, Ready to File, Filed, Completed, Overdue and Disputed/Needs Review.
- Store rule effective dates, official reference URL, last verified date, verified-by user and rule version.
- Generate recurring periods only according to configured rules and avoid duplicate obligation generation.
- Provide calendar, list and entity-centric views; filter by period, service, owner, status and due-date window.
- Configure reminders before due date, on due date and after due date; create escalation tasks and record delivery outcomes.

- Allow authorised manual overrides with mandatory reason, prior value and audit event.
- Do not assume an obligation applies to every entity of a given type; applicability must be explicitly determined or reviewed.
- Link filing proof, acknowledgement/reference number, return copy, challan or other evidence to the obligation.
- Allow rule updates to affect future obligations without silently changing already-filed records.

Acceptance criteria

- No statutory date is activated in production without a verified source and review owner.
- Recurring generation is idempotent and does not create duplicates.
- Overdue reporting distinguishes legal due dates from internal targets.

Module 9. Billing, payments, expenses and receivables

Purpose: Track amounts quoted, invoiced, collected, allocated, refunded and outstanding.

Functional requirements

- Create invoices with unique numbering, issue/due dates, customer details, line items, tax treatment, discounts, notes and payment terms.
- Support professional fees, statutory/government fees, third-party charges and reimbursable expenses as distinct line items/categories.
- Support configured GST/tax rates and place-of-supply/business rules only after finance review; tax behaviour must not be hardcoded without configuration.
- Record payments via cash/bank transfer/payment gateway with reference number, date, amount, method and reconciliation status.
- Allocate a payment across one or more invoices; support partial payment, advance, overpayment and unapplied credit.
- Calculate invoice balance from posted transactions; do not rely on manually edited outstanding totals.
- Support credit notes, refund requests, approval thresholds, refund execution records and reversals with audit trail.
- Record business expenses and attach receipts; distinguish pass-through client disbursements from consultancy operating expenses.
- Provide receivables ageing, client statement, payment history, collection follow-up and exportable finance reports.
- Support cancellation/voiding only under controlled rules; issued financial records should not be silently deleted.
- Keep this module as operational billing/receivables, not a full general ledger unless separately scoped.

Acceptance criteria

- All monetary calculations use integer minor units or fixed-precision decimals, never binary floating point.
- Payment allocations and invoice balances reconcile after partial payments, credit notes and refunds.
- Concurrent payment recording cannot double-allocate funds or duplicate transactions.
- Invoice numbering is unique and issued financial records retain an audit history.

Module 10. Communication centre and notification automation

Purpose: Send traceable client and internal communications through configured providers.

Functional requirements

- Support email first; add SMS and WhatsApp Business through provider adapters.
- Maintain templates for welcome messages, document requests, quotation follow-up, payment reminders, case updates, rejection/correction requests, completion and renewal.
- Support variables with validation and safe escaping; preview templates before activation.

- Track recipient, channel, template/version, queued/sent/delivered/failed status where provider data exists, provider reference and timestamps.
- Keep internal notes separate from messages intended for clients.
- Use consent/opt-out rules and applicable provider/platform policies for promotional communications.
- Queue messages asynchronously with retry/backoff, dead-letter handling and idempotency keys.
- Do not include sensitive identity details or public file links in ordinary messages.
- Allow scheduled messages to be cancelled when a case/obligation is completed or the underlying event changes.
- Record message history against the relevant client, entity, case or invoice.

Acceptance criteria

- Provider failure does not block case updates or transaction commits.
- Retry logic avoids sending duplicates unnecessarily.
- Users can distinguish a queued message from one confirmed as delivered.

Module 11. Employee operations, workload and approvals

Purpose: Help managers allocate work and ensure appropriate review.

Functional requirements

- Maintain teams/departments, reporting lines, skill/service tags and active availability.
- Provide personal task queue, team queue, overdue list and workload by employee/service.
- Support task priorities, due dates, dependencies, reassignment and escalation reason.
- Configure maker-checker approval for selected service steps, document reviews, refunds and high-risk changes.
- Prevent self-approval where policy requires separation of duties.
- Provide correction/rework loops with reasons and reviewer feedback.
- Track internal SLA adherence, completion time, ageing, reassignment count and rework rate.
- Do not treat raw task count as the sole employee performance metric; allow managers to interpret complexity and workload.

Acceptance criteria

- A manager can identify unassigned and overdue work quickly.
- Approvals record approver, decision, timestamp and comments.
- Approval permissions are enforced by the backend.

Module 12. Dashboards, reports and exports

Purpose: Provide role-appropriate operational and financial visibility.

Functional requirements

- Owner dashboard: active cases, overdue work, upcoming obligations, pending documents, new leads, conversion, collections and revenue.
- Operations dashboard: cases by status/service/assignee, case ageing, workload, SLA risk, rejection and rework.
- Sales dashboard: lead source, stage, follow-up overdue, quote conversion and service demand.
- Finance dashboard: invoiced, collected, outstanding, overdue ageing, refunds and expenses.
- Compliance dashboard: due in selected windows, overdue obligations, missing filing evidence and rule-review status.
- Support date ranges, entity/service/assignee/status filters and drill-down to authorised records.

- Export CSV/XLSX/PDF where appropriate; exports must follow the same permissions and avoid exposing unnecessary sensitive data.
- Clearly define metrics, date ranges and whether values are invoice-based, cash-based or operational counts.
- Large reports should run asynchronously and notify the user when ready.

Acceptance criteria

- Dashboard totals reconcile to their underlying records and documented metric definitions.
- Exports respect user permissions and filters.
- Reports do not expose data across unauthorized client/entity boundaries.

Module 13. Administration, configuration and audit

Purpose: Allow safe configuration and preserve accountability for important actions.

Functional requirements

- Configure organisation profile, business timezone, date formats, currency, numbering sequences and notification defaults.
- Manage service categories, templates, custom fields, status labels and escalation rules through controlled administration.
- Maintain append-only audit events for sensitive or material actions: actor, timestamp, action, resource, outcome, request/correlation ID and safe before/after metadata.
- Audit login/security events, role changes, document access, case transitions, invoice issue/adjustment, payment allocation, refund approval and compliance rule changes.
- Avoid storing passwords, access tokens, full identity documents or other secrets in audit payloads.
- Provide searchable audit views restricted to authorised roles; normal users must not be able to modify audit history.
- Support archival, retention and deletion workflows with documented approvals and legal holds.
- Track integration configuration health without exposing secret credentials.

Acceptance criteria

- Audit records cannot be changed through ordinary application APIs.
- Material updates are attributable to an actor or explicitly identified system job.
- Secrets are redacted from logs and audit payloads.

Module 14. External integrations and provider adapters

Purpose: Integrate external systems without coupling core business logic to one provider.

Functional requirements

- Payment gateway adapter with verified webhooks, signature verification, idempotency and reconciliation.
- Email/SMS/WhatsApp provider adapters with status callbacks and provider error mapping.
- Private object storage adapter for upload/download, retention and access control.
- Optional e-signature provider integration with signer identity, document version, status and completion evidence.
- Optional calendar and accounting export integrations, scoped by an approved use case.
- Store integration name, status, configuration metadata, last health check and safe error details; encrypt secrets in an appropriate secret store.
- Use official APIs and documented permissions. Do not assume government portal APIs exist or are available to the business.
- For unavailable/unauthorised integrations, support manual reference entry and evidence upload.
- Verify webhook signatures, reject replayed/duplicate events and log processing outcomes.
- Provide health checks, timeout handling, retries and alerting for critical integrations.

Acceptance criteria

- A provider outage does not corrupt case or finance data.
- Webhook retries are idempotent.
- Integration secrets never appear in the UI, source control or ordinary logs.

Module 15. Knowledge base, SOPs and template library

Purpose: Give employees approved, versioned instructions for consistent service delivery.

Functional requirements

- Store service SOPs, internal notes, client instruction sheets, checklists, document templates and standard response templates.
- Track version, owner, reviewer, approval status, effective date and next review date.
- Link guidance to services, case tasks and compliance rules.
- Maintain official reference URLs and last-verified dates for statutory guidance.
- Support draft/review/approved/retired states and preserve old versions for historical reference.
- Restrict internal procedures from client portal access unless explicitly published as client-facing content.
- Do not allow AI-generated or unreviewed legal text to become an approved template automatically.

Acceptance criteria

- A case can identify which approved template/SOP version was used.
- Retired guidance is not silently presented as current.

Module 16. Client self-service portal

Purpose: Allow clients and authorized representatives to complete simple tasks without staff intervention.

Functional requirements

- Secure invitation and login with account recovery and session revocation.
- Show only the client’s authorized entities, service cases, requests, due dates, invoices and deliverables.
- Allow document upload against a specific request/checklist item and show review status and correction reason.
- Allow quotation review/acceptance where the business process supports electronic acceptance.
- Show case progress and approved client-visible updates without revealing internal notes or staff-only information.
- Support invoice downloads, payment links and payment history where integrated.
- Support authorized representative management and revocation.
- Provide a support/enquiry form with case linking and anti-abuse controls.
- Make portal features configurable and keep the initial release focused on internal operations if time is constrained.

Acceptance criteria

- Cross-client access attempts are denied and security-tested.
- Internal notes and restricted documents never appear in client responses.
- Uploads are scanned and reviewed before being treated as accepted.

6. Service catalogue and initial service templates

The following are suggested starting categories. The business owner and qualified compliance reviewer must approve the actual catalogue, scope, pricing, eligibility rules and service checklists.

Category	Example services	Typical workflow-specific data
Business formation	Private Limited Company, LLP, OPC, partnership/proprietorship setup assistance	Proposed name, entity type, promoters/members, registered office, approvals, application references, certificates.
Tax registration	GST registration/amendment/cancellation, PAN-related assistance	Business activity, jurisdiction, address proof, tax identifiers, application status and acknowledgement.
Corporate compliance	ROC annual/event-based filings, director/KYC compliance, office changes	Entity identifier, financial year/event date, applicable forms, signatories, filing evidence and fees.
Intellectual property	Trademark search assistance, application filing, objection response, renewal tracking	Mark, class, owner, goods/services, filing date, application number, examination/objection dates and evidence.
Tax and statutory support	GST return assistance, income-tax return assistance, TDS-related services	Period, applicable forms/returns, source records, reviewer approval, payment/challan and submission proof.
Licences and registrations	Udyam/MSME, IEC, professional tax and other applicable licences	Eligibility, jurisdiction, identifiers, supporting documents, validity and renewal conditions.
Retainers and recurring support	Annual compliance package, monthly bookkeeping coordination, advisory retainer	Covered services, exclusions, billing cycle, included volume, renewal date and service-level expectations.

Service template fields

- Service code, name, category, version, active dates and service owner.
- Eligibility and applicability checklist, jurisdiction and exclusions.
- Required client/entity fields and required documents, including whether each item is conditional.
- Task sequence, role assignment, dependencies, reviewer gates and target durations.
- Client approvals/declarations and professional authorization requirements.
- Fee components, tax configuration reference, government-fee handling and payment milestones.
- Possible external outcomes, correction loops, closure criteria and final deliverables.
- Recurring obligation generation rules, if applicable, with verified sources and rule version.

Example: company incorporation workflow

- Lead qualified and scope/quotation accepted.
- Case created and relevant business entity/promoter records established.
- Checklist generated; required identity/address and entity information collected.
- Documents checked for completeness and consistency; corrections requested when needed.
- Application package prepared and internally reviewed; required client authorisations recorded.
- Submission made by an authorised person through the applicable permitted channel; reference and timestamp stored.
- External response tracked; correction/resubmission loop supported without deleting prior history.
- Approval evidence and deliverables uploaded; client handover recorded; billing reconciled.
- Applicable post-incorporation obligations generated only after rule/applicability review.

7. End-to-end workflows and state transitions

Lead-to-service workflow

- Create lead → record contact/follow-up → qualify → prepare quotation → accept/reject/expire → convert accepted quotation → create service order/case → assign owner and reviewer.

Case delivery workflow

- New → Awaiting Client / Awaiting Payment / In Preparation → Documents Under Review → Internal Review → Ready to Submit → Submitted → Awaiting Authority → Approved/Correction Required → Delivered → Completed.
- A service may use only a subset of these states. Configure allowed transitions and guards per service template.
- Correction Required may return to document collection, preparation or internal review; previous submissions and feedback must remain in the timeline.
- On Hold and Cancelled require a reason. Reopening a completed case requires permission and a reason.
- A status is not proof of government approval. External outcome and supporting evidence must be recorded separately.

Obligation workflow

- Rule reviewed → applicability assessed → obligation created for entity/period → owner assigned → reminders sent → evidence prepared → filing/submission recorded → evidence reviewed → completed.
- Not Applicable requires reason, actor and timestamp. It must not be used to hide an obligation that is merely pending assessment.
- Rule changes must be versioned; already-filed obligations retain historical rule references.

Finance workflow

- Quotation → accepted order → invoice according to policy → payment received → payment allocated → balance recalculated → receipt/statement available.
- Refund request → approval based on permission/threshold → refund execution or recorded external refund → reconciliation.
- Webhook and retry handling must be idempotent; duplicate provider callbacks must not create duplicate payments.

8. Data model and database requirements

Use PostgreSQL with Prisma migrations. The following table groups are required design candidates; the implementation agent may refine table names and split tables where justified, but must preserve the relationships and business rules.

Domain	Core entities / tables
Identity & access	users, roles, permissions, user_roles, role_permissions, sessions, auth_events
CRM	leads, clients, contacts, activities, follow_ups, lead_sources, client_tags
Entities	business_entities, entity_members, entity_addresses, entity_identifiers, entity_authorisations
Catalogue & sales	service_categories, services, service_versions, service_prices, service_templates, quotations, quotation_items, service_orders, order_items
Case operations	cases, case_assignments, case_tasks, task_dependencies, case_comments, case_status_history, approvals
Documents	documents, document_versions, document_requirements, document_reviews, document_access_logs
Compliance	compliance_rules, compliance_rule_versions, compliance_obligations, filing_records, renewals
Finance	invoices, invoice_items, payments, payment_allocations, credit_notes, refunds, expenses, receipts
Communications	notification_templates, notification_jobs, message_logs, consent_records, provider_webhooks
Platform	audit_logs, system_settings, integration_accounts, idempotency_keys, background_job_records

Database implementation rules

- Use UUID primary keys; use separate human-readable unique reference numbers for cases, invoices, quotations and obligations.
- Add created_at, updated_at, created_by and updated_by where appropriate; use deleted/archived fields only when consistent with retention and audit requirements.

- Use foreign keys, unique constraints, check constraints and indexes to enforce data integrity.
- Use transactions for multi-record business operations such as order conversion, invoice issue and payment allocation.
- Represent money with integer minor units or fixed-precision decimal columns and an explicit currency code.
- Store statutory date-only values as date types where suitable; use UTC timestamps for events and configure display timezone.
- Version templates, service prices and compliance rules; historical records must reference the version actually used.
- Keep binary files outside PostgreSQL unless a documented requirement justifies otherwise; store metadata and storage keys in the database.
- Use row/object-level authorization in application logic; consider PostgreSQL row-level security only if implemented and tested consistently.
- Avoid unbounded JSON blobs for core entities. JSON may be used for versioned configuration or service-specific data with validation and documentation.

9. API and integration standards

API conventions

- Use versioned REST routes such as /api/v1/clients and /api/v1/cases/:id/transitions.
- Use DTO validation and reject unknown/invalid fields where practical; return consistent error shapes with a request/correlation ID.
- Use pagination, sorting and filtering for list endpoints; apply server-side limits and indexed query patterns.
- Use explicit workflow commands for transitions rather than unrestricted status updates.
- Document endpoints, request/response schemas, authentication, permissions and error codes in OpenAPI/Swagger.
- Use idempotency keys for operations that may be retried, especially payment webhooks, quotation conversion and financial commands.
- Do not expose internal database errors, stack traces, secret values or sensitive identifiers in client-facing responses.

Resource	Example endpoint patterns
Auth	POST /auth/login · POST /auth/logout · POST /auth/refresh · POST /auth/password-reset
Clients	GET/POST /clients · GET/PATCH /clients/:id
Entities	GET/POST /entities · GET/PATCH /entities/:id
Services	GET /services · POST /services · PATCH /services/:id
Quotations	POST /quotations · POST /quotations/:id/send · POST /quotations/:id/accept
Cases	POST /cases · GET /cases/:id · POST /cases/:id/transitions · POST /cases/:id/assign
Tasks	GET /tasks/my-queue · PATCH /tasks/:id · POST /tasks/:id/complete
Documents	POST /documents/upload-url · POST /documents/:id/review · GET /documents/:id/download-url
Compliance	GET /compliance/calendar · POST /obligations · POST /obligations/:id/filing-records
Finance	POST /invoices · POST /payments · POST /payments/:id/allocations · GET /reports/receivables
Reports	GET /reports/operations · GET /reports/revenue · GET /reports/compliance
Administration	GET /audit-logs · POST /users · PATCH /roles/:id

Background jobs

- Use Redis/BullMQ or equivalent for scheduled reminders, recurring obligation generation, notification delivery, report generation and document scanning.
- Jobs must be retryable, observable and idempotent. Use bounded retries with backoff and a dead-letter/failure review path.
- Store enough job metadata to diagnose failures without placing sensitive document contents or credentials in job payloads.

- Do not rely on a browser tab or a single in-memory timer for critical reminders.

10. Security, privacy and audit requirements

Minimum controls

- TLS/HTTPS everywhere; secure cookie/session configuration; password hashing; rate limiting; secure headers and dependency scanning.
- Least-privilege RBAC with record-level authorization for client, entity, case, invoice and document resources.
- Private file storage, short-lived signed links, malware scanning, MIME/type and size checks, safe filenames and access logging.
- Encrypt sensitive data at rest where supported; protect database backups and object storage; keep secrets in environment/secret management, never source control.
- Redact credentials, tokens, full PAN/Aadhaar and document contents from application logs, traces and audit metadata.
- Audit privileged actions, access to sensitive documents, status transitions, approvals, financial changes and compliance rule updates.
- Support data retention schedules, archival, correction/export requests and secure deletion according to legal review and any applicable retention duties.
- Maintain incident response procedures, access reviews, backup monitoring, restore drills and vulnerability remediation.
- Review privacy notices, lawful purpose, consent where required, processor/vendor terms, breach handling and applicable Indian data-protection obligations before production.

The implementation must not assume that storing a client's portal credentials is necessary. Prefer client-controlled access, authorised filing, official APIs where available, or manual tracking with evidence. Do not build unsupported scraping or unauthorised portal automation.

11. Reporting and dashboard requirements

Dashboard	Required indicators
Owner	Active cases, cases at risk, overdue tasks, obligations due soon, new leads, quotation conversion, invoiced, collected, outstanding, expenses and revenue trends.
Operations	Cases by service/status/owner, unassigned tasks, ageing, pending client inputs, review queue, rejection/rework and internal SLA performance.
Sales	Leads by source/stage/owner, overdue follow-ups, quotations sent/accepted/lost, conversion rate and service demand.
Finance	Invoices issued, collections, outstanding balances, ageing buckets, unapplied payments, refunds, expenses and client statements.
Compliance	Obligations due in selected periods, overdue items, not-assessed obligations, missing filing evidence, rule verification status and assigned owner.
Employee	My tasks, due today, overdue tasks, pending review, waiting-for-client items and upcoming assignments.

Metric definitions

- Document the formula, time window, included statuses and accounting basis for every dashboard metric.
- Separate billed revenue from cash collected and distinguish client pass-through disbursements from professional fees.
- Define case ageing consistently (e.g. opened date to current date, with paused time treatment documented).
- Allow drill-down only to records the user is authorized to view.
- Ensure exported reports apply the same permission checks and filters as on-screen results.

12. Non-functional requirements

Area	Requirement
Maintainability	Modular monolith with clear domain modules, dependency boundaries, typed DTOs, code conventions and documented business rules.
Performance	Set measurable targets during discovery. Initial baseline: common list/detail APIs should typically respond within 500 ms at expected pilot load, excluding external provider latency.
Reliability	Health/readiness checks, structured logs, error monitoring, retryable background jobs and graceful handling of provider outages.
Scalability	Stateless API where practical; pagination and indexes; queue-based background work; object storage for files.
Accessibility/usability	Responsive desktop-first admin UI, keyboard-accessible controls, clear validation, loading/empty/error states and confirmation for destructive actions.
Data integrity	Foreign keys, unique constraints, transactions and idempotency for critical multi-step operations.
Observability	Correlation IDs, structured logs, queue health, error rates, latency and provider failure visibility without logging secrets.
Backup/recovery	Automated encrypted backups; documented retention and recovery objectives; regular restoration tests.
Compatibility	Support current stable desktop browsers used by the business; define supported versions before release.
Localization	Indian business context, INR by default, configurable timezone, date formats and locale-aware currency formatting.
Deployment	Separate development, staging and production environments; environment-specific secrets; reviewed database migrations and rollback plan.
Testing	Unit, integration, API, end-to-end, authorization, financial edge-case and backup/restore tests.

13. Testing and acceptance criteria

Mandatory test suites

- Unit tests for pricing, tax configuration, date calculations, workflow transition rules, permission checks and financial balances.
- Integration tests for database constraints, transaction boundaries, document metadata/storage operations, queue processing and webhook verification.
- End-to-end tests for lead conversion, quotation acceptance, case execution, document rejection/re-upload, obligation completion, invoice/payment allocation and refund approval.
- Authorization tests for every role and for cross-client/entity/case access attempts.
- Security tests for IDOR, privilege escalation, file upload abuse, expired links, injection, rate limits and sensitive-data leakage.
- Reliability tests for duplicate webhook delivery, queue retries, provider timeouts, job restart and recovery after deployment.
- Financial tests for partial payments, multiple invoice allocations, overpayment, credit notes, refund/reversal, rounding and concurrent updates.
- Operational tests for backup restore, migration on a copy of production-like data, deployment rollback and log redaction.

Release acceptance gates

- No unresolved critical/high security issues.
- All P0 workflows work end to end using persisted database records.
- Every protected API endpoint has documented authorization expectations and tests.
- Financial totals reconcile against source transactions; invoice numbering and payment handling are tested.
- Compliance rules in production are verified by the designated reviewer and reference current official sources.
- Backup restoration and production deployment have been rehearsed.

- Business users complete a user acceptance test using representative service cases.
- Runbook covers user administration, failed jobs, provider outages, backup restore, incident response and release/rollback.

14. Implementation phases and priority

Phase	Indicative timing	Deliverables
1. Discovery	Weeks 1–2	Approved requirements, user journeys, wireframes, service templates, data model, architecture, repository and CI.
2. Foundation + CRM	Weeks 3–4	Authentication, RBAC, client/entity records, leads, service catalogue and pricing.
3. Service operations	Weeks 5–7	Quotations, orders, cases, tasks, documents, checklists, review and audit timeline.
4. Compliance + notifications	Weeks 8–9	Obligation rules, calendar, reminders, escalation, email integration and job monitoring.
5. Finance + reporting	Weeks 10–11	Invoices, payment allocation, receivables, expenses and operational/finance dashboards.
6. Hardening	Weeks 12–13	Security, permissions, financial edge cases, end-to-end tests, backup/restore and UAT.
7. Pilot + launch	Weeks 14–16	Data import, approved templates, staff training, pilot, fixes, production rollout and runbooks.

These timings are indicative and depend on staffing, quality expectations, existing assets and the number of service templates. Do not sacrifice authorization, data integrity, backup/recovery or financial correctness to meet a calendar estimate.

Recommended MVP

- P0: Authentication/RBAC; CRM; business entities; service catalogue; quotations/orders; case/task workflow; document vault/checklists; compliance calendar; invoices/payments/receivables; core dashboard; audit trail; backups.
- P1: WhatsApp/SMS, advanced approvals, e-signature, accounting/calendar integration, richer knowledge base and expanded reports.
- P2: Client self-service portal, advanced analytics, AI-assisted drafting/search and broader automation—only with human review and suitable safeguards.

15. Definition of done

A module is done only when all applicable points below are met:

- Requirements and edge cases are documented; acceptance criteria are met.
- Frontend screens use real API operations and persisted records; no production-critical workflow depends on mock data.
- Backend DTO validation, authorization, object-level access checks and safe error handling are implemented.
- Database migrations, constraints, indexes and seed/demo data are committed and documented.
- Unit/integration/end-to-end tests cover normal paths, invalid states, permission failures and material edge cases.
- Audit events are written for critical changes without exposing secrets or sensitive content.
- Loading, empty, validation, success and error states are handled in the UI.
- API docs, environment variables, setup instructions and relevant runbooks are updated.
- The feature is demonstrated in staging and accepted by the designated business owner/reviewer.

Appendix A. Suggested repository structure

```
roy-consultancy/
apps/
web/ # Next.js admin and employee UI; client portal may be added later
api/ # NestJS REST API
worker/ # Optional separate process for background jobs
packages/
ui/ # Shared UI components
types/ # Shared API types where appropriate
config/ # Lint, TypeScript and test configuration
prisma/
schema.prisma
migrations/
seed.ts
docs/
requirements/
architecture/
api/
runbooks/
tests/
e2e/
security/
infra/
docker/ # Optional container definitions
deployment/
.env.example
README.md
```

Engineering conventions

- Use a package manager and workspace layout consistently; do not duplicate domain logic between the web app, API and worker.
- Keep business logic in domain/application services, not React components or controller methods.
- Use Prisma migrations as the source of truth for schema changes.
- Provide .env.example with placeholder names only; never commit actual secrets.
- Use code formatting, linting, type checking and tests in CI before merging.
- Keep production, staging and development credentials/data isolated.

Appendix B. Agent handoff checklist

Give this PDF to the coding agent together with the following instruction:

> Build the Roy Consultancy Operations Platform according to this Software Requirements Specification. First inspect the repository and report the current state. Then produce an implementation plan mapping every module to routes/screens, API endpoints, database models, permissions, tests and milestones. Identify unresolved decisions instead of inventing legal/compliance rules. Implement in small vertical slices with working persistence, migrations, tests and documentation. Prioritize P0 modules. Use Next.js, NestJS, TypeScript, PostgreSQL and Prisma unless the existing repository has a documented, approved alternative. Use a modular monolith. Never use mock data for production workflows. Do not claim government filing integrations unless they are verified and authorized. Keep statutory rules configurable and require human verification. After each milestone, report completed features, tests run, known gaps, migration/setup steps and the next milestone.

Before coding begins, confirm

- Who will own product decisions and approve service workflows?
- Which 5–10 services must be operational in the first pilot?

- Which invoice numbering, tax, advance-payment and refund policies does the business use?
- Which staff roles need access to each record type and sensitive document class?
- Which official sources and qualified reviewers will maintain compliance rules?
- Where will the application, database, Redis and file storage be hosted, and who owns backups and incident response?

End of specification

Document owner: Roy Consultancy. This document should be version-controlled and updated whenever the approved product scope or business rules change.